

Corporate Banking Loan Pricing & Deal Approval System

Capstone Project 3: Modernizing credit workflows for mid-market and enterprise clients.



Angular 18



Spring Boot 3.x



MongoDB



Docker

Key Context: A secure, role-based Single Page Application (SPA) designed to centralize loan lifecycles, enforce consistent pricing, and ensure immutable audit trails.

Moving Beyond Spreadsheets and Email Chains

Legacy Workflow



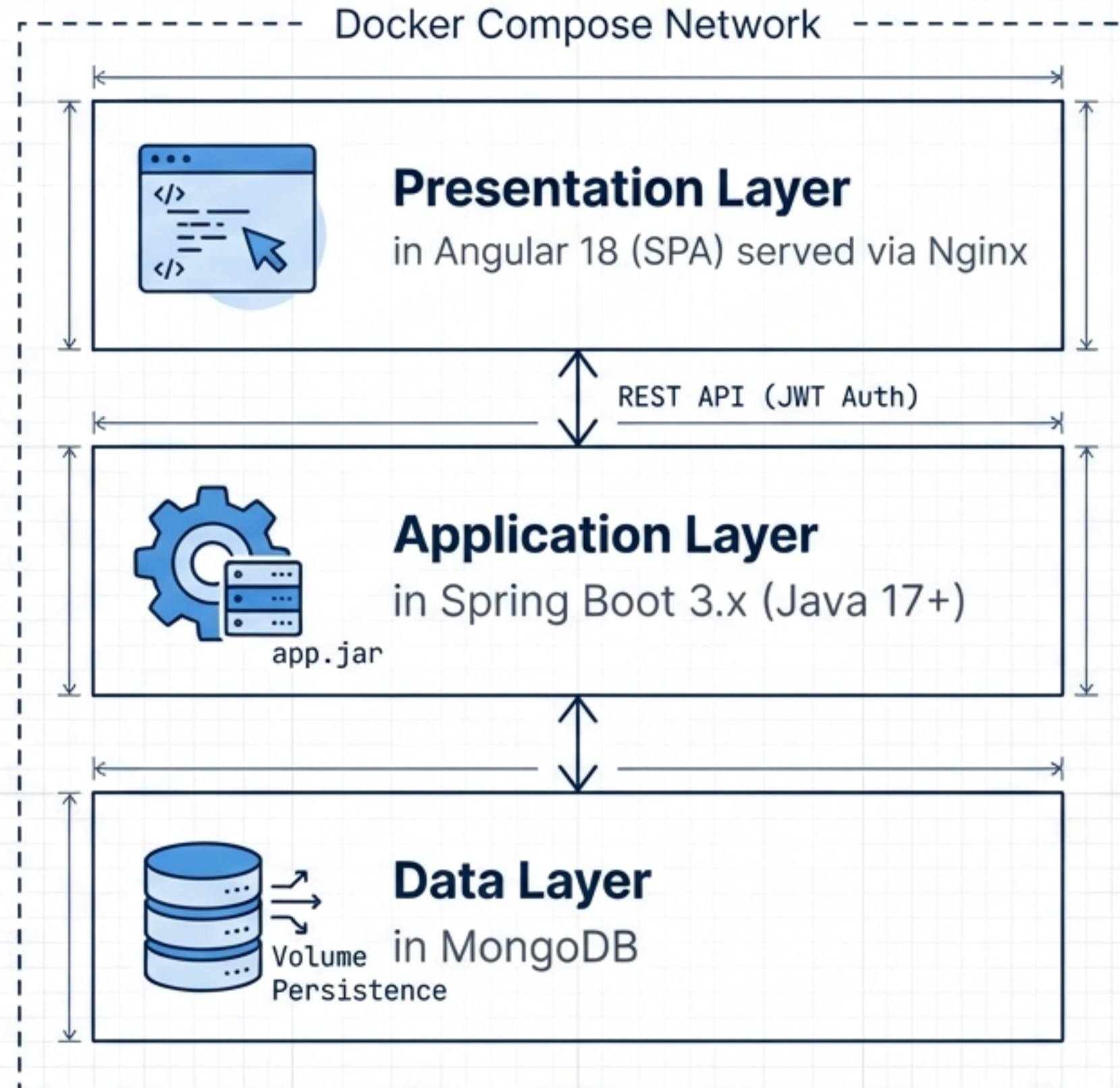
- **Inconsistent Pricing:** Calculations vary between Relationship Managers.
- **No Audit Trail:** Impossible to track who edited a deal or when.
- **Security Risks:** Unauthorized edits on sensitive fields and lack of visibility.

The System Blueprint



- **Centralization:** A single source of truth for all deal requests.
- **Enforcement:** Hard-coded logic for pricing and role-based permissions.
- **Compliance:** A “deleted” flag implementation (Soft Delete) ensuring data permanence.

A Containerized, Three-Tier Architecture



Rigid Separation of Duties: RBAC Implementation

The application enforces a strict firewall between Origination and Approval.

RELATIONSHIP MANAGER (User)

Origination & Pricing

- ✓ Create requests
 - ✓ Run pricing formulas
 - ✓ Update drafts
-
- 🔒 Blind to sensitive 'Sanctioned' data.

CREDIT MANAGER (Admin)

Risk & Compliance

- ✓ Approve / Reject
 - ✓ Modify Sanctioned Amounts
 - ✓ Soft Delete records
-
- 👁️ Full visibility into Audit History.

Authentication Mechanics & User Schema

Security Specs

-  **Hashing:** BCrypt for password storage
-  **Session:** Stateless JWT authentication (Access tokens + expiry)
-  **Route Protection:** Angular AuthGuard & RoleGuard

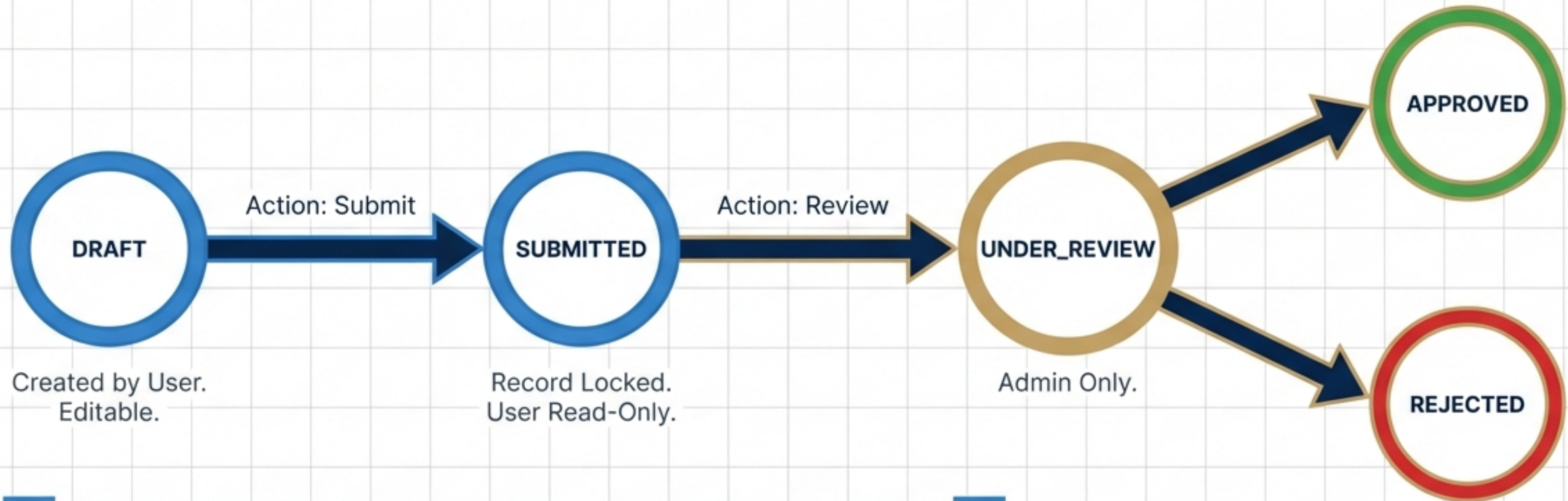
Key API Endpoints

- **POST** /api/auth/login (Public)
- **GET** /api/users/me (Context)
- **PUT** /api/admin/users/{id}/status (Activation)

users.collection

```
{
  "_id": "ObjectId('...')",
  "email": "rm@bank.com",
  "password": "$2a$10$...", // Hashed
  "role": "USER", // ENUM: USER | ADMIN
  "active": true,
  "createdAt": "2023-10-27T10:00:00Z"
}
```


The Loan Lifecycle State Machine



Logic Constraints

- Users can only edit when status is DRAFT.
- Only Admins can move a loan from SUBMITTED to UNDER_REVIEW.
- Sensitive fields are strictly read-only for Users regardless of status.

Data Modeling: The Loan Collection

```
{
  "_id": "ObjectId",
  "clientName": "OmniTech Pvt Ltd",
  "financials": { "revenue": 120M, "ebitda": 14M },
  "status": "SUBMITTED",
  // Sensitive Admin Fields
  "sanctionedAmount": null,
  "approvedInterestRate": null,
  // The Audit Trail
  "actions": [
    {
      "by": "ObjectId(User)",
      "action": "SUBMITTED",
      "comments": "Docs verified",
      "timestamp": "ISODate"
    }
  ]
}
```

Nested document for
business data

Field-level security
(Admin access only)

Immutable history of all
transitions

API Strategy: Loan Management Endpoints

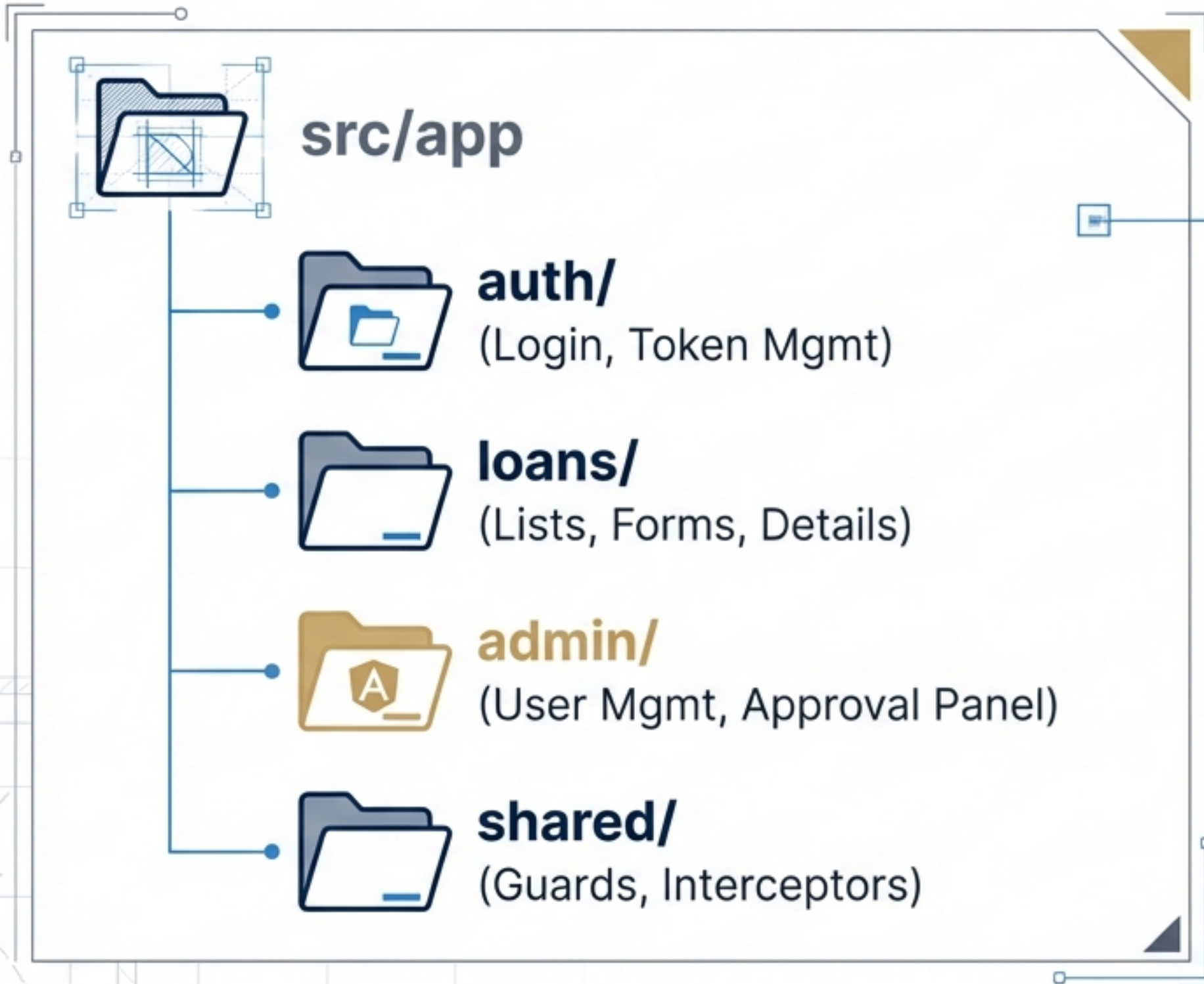
Method	Endpoint	Description
POST	/api/loans	Initialization (Draft Creation)
GET	/api/loans	List view with server-side pagination & sorting
PATCH	/api/loans/{id}/status	State transition trigger (Submit/Approve/Reject)
DELETE	/api/loans/{id}	Soft Delete (sets deleted: true)

Validation & Error Handling

```
{  
  "timestamp": "...",  
  "status": 400,  
  "error": "Bad Request",  
  "message": "Validation failed"  
}
```

Bean Validation enforces field-level rules. Errors return in standardized JSON format.

Angular Frontend Architecture



Role-Based UI Logic

Conditional Rendering

Approval Panel component only loads for ADMIN role.

Field Locking

Inputs disabled if User viewing sensitive fields or status $\neq$ DRAFT.

Filtering

Deleted loans hidden from Users.

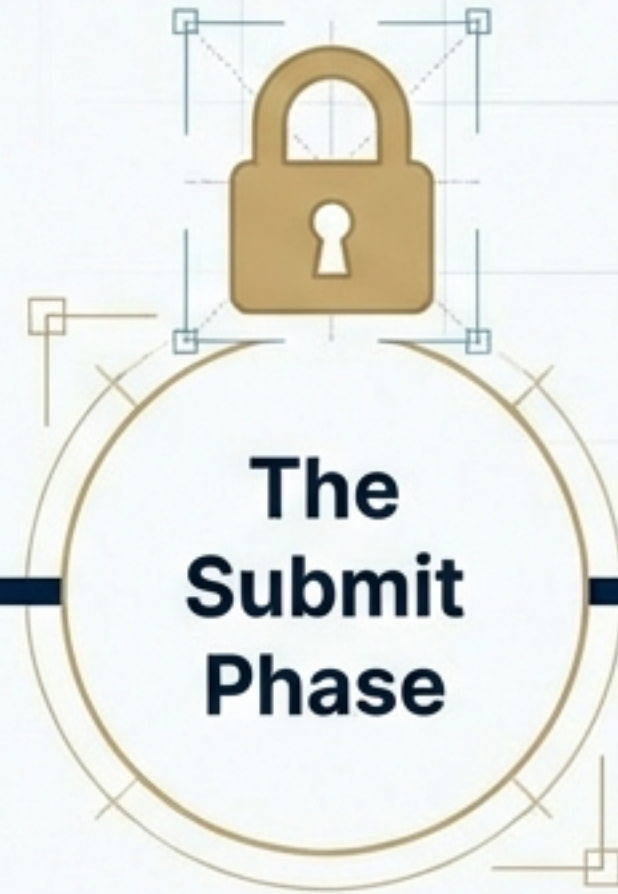
Workflow A: The Relationship Manager



Input Client Name, Loan Type, and Loan Type, and Financials (Revenue, EBITDA).

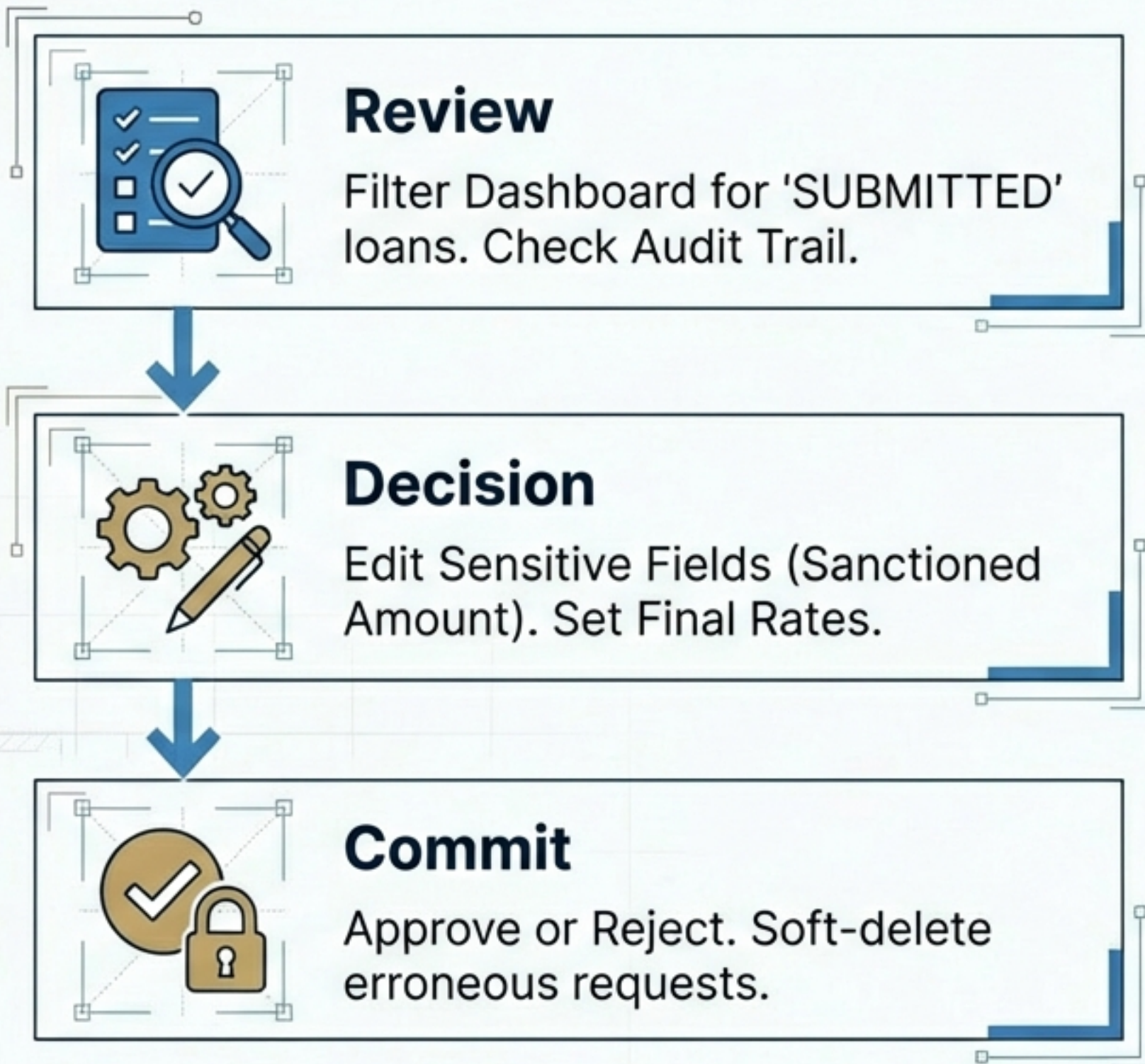


Trigger pricing calculation. System computes formula based on inputs.



Transition status DRAFT → SUBMITTED. Record becomes Read-Only.

Workflow B: The Credit Manager



Approval Panel

Credit Decision

Sanctioned Amount

JetBrains Mono

Final Rate

JetBrains Mono

APPROVE

REJECT

Backend Engineering Standards



Data Integrity

Use of DTOs to decouple internal schema from API. Global Exception Handling catches validation errors.

Validation

Bean Validation ensures data quality before it reaches the service layer.

Quality Assurance Strategy

Testing Pyramid



Backend: JUnit + Mockito

Target: 80% Coverage.

Testing Service-layer logic and Controller endpoints. Integration testing via Testcontainers.

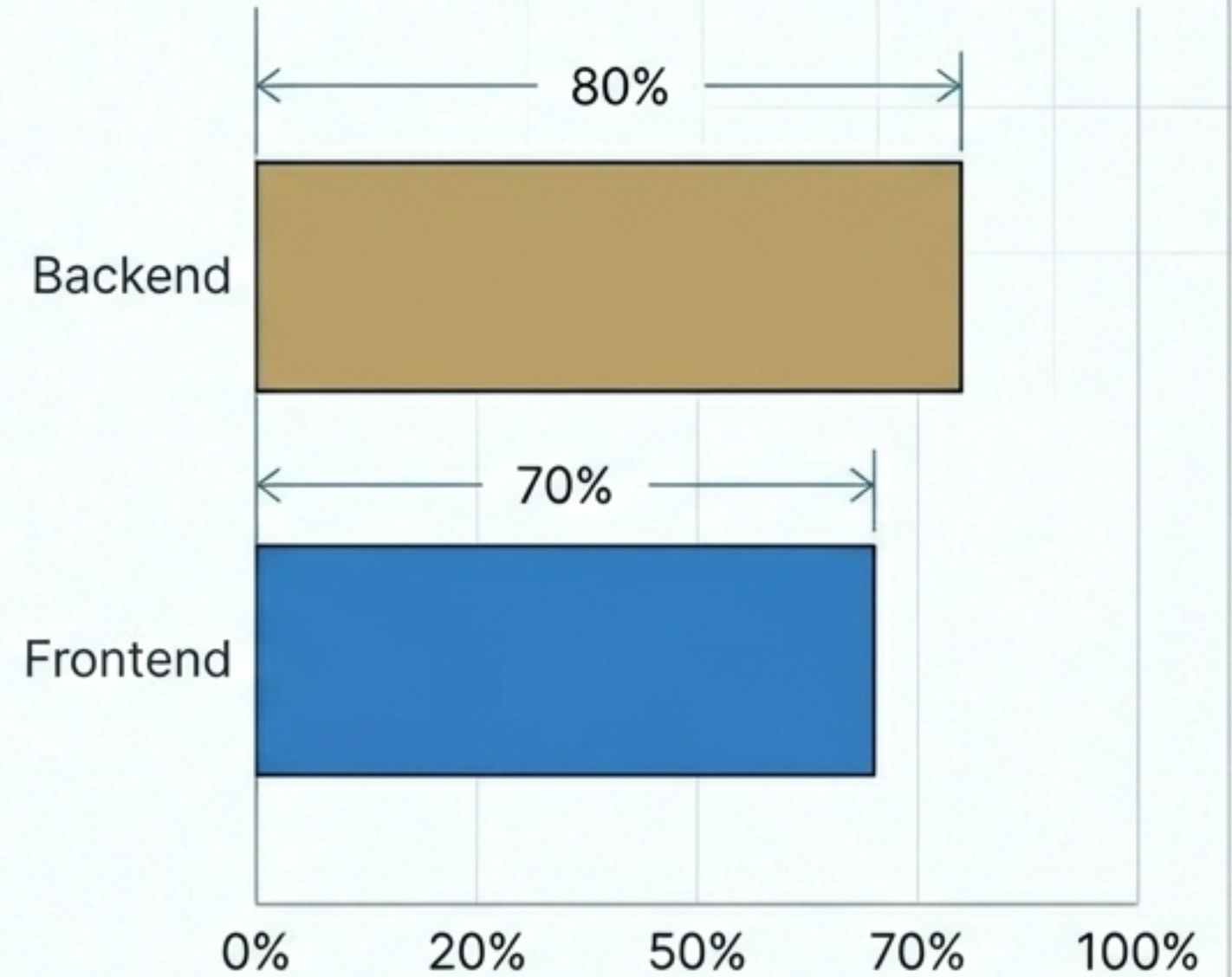


Frontend: Jasmine + Karma

Target: 70% Coverage.

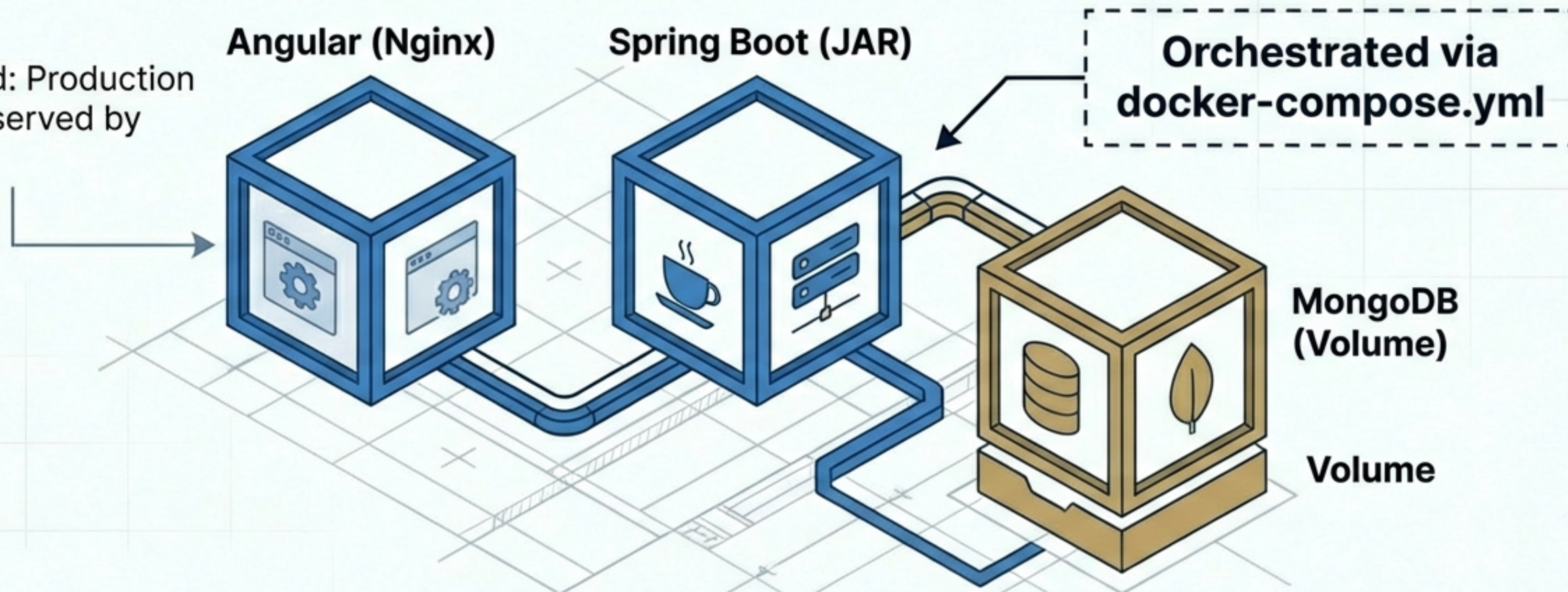
Testing Form validation logic, Route Guards, and Service integration.

Coverage Targets



Deployment & Containerization

- Frontend: Production artifact served by Nginx.

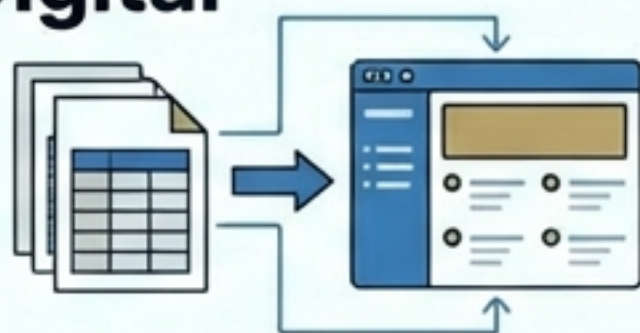


- **Frontend:** Production artifact served by Nginx.
- **Backend:** OpenJDK environment.
- **Database:** Persistent volume mapping.
- **Outcome:** Reproducible, environment-agnostic deployment.

Project Impact Summary

From Manual to Digital

Migrated from spreadsheet dependency to a structured SPA.



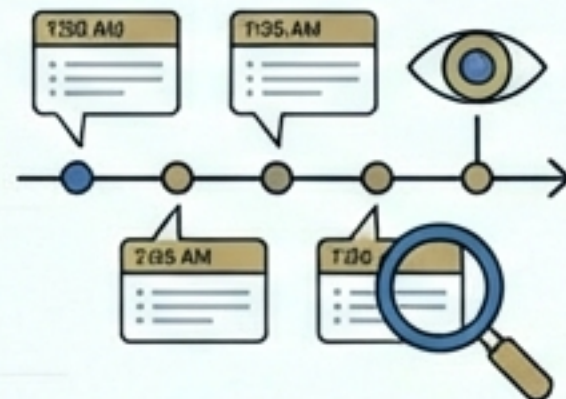
Security Achieved

Full RBAC with JWT and granular field-level security.



Visibility Achieved

Permanent, immutable audit history for every deal.



Ready for Scale

Delivered on a containerized, cloud-ready stack.

